

2026

NIGERIA PMC

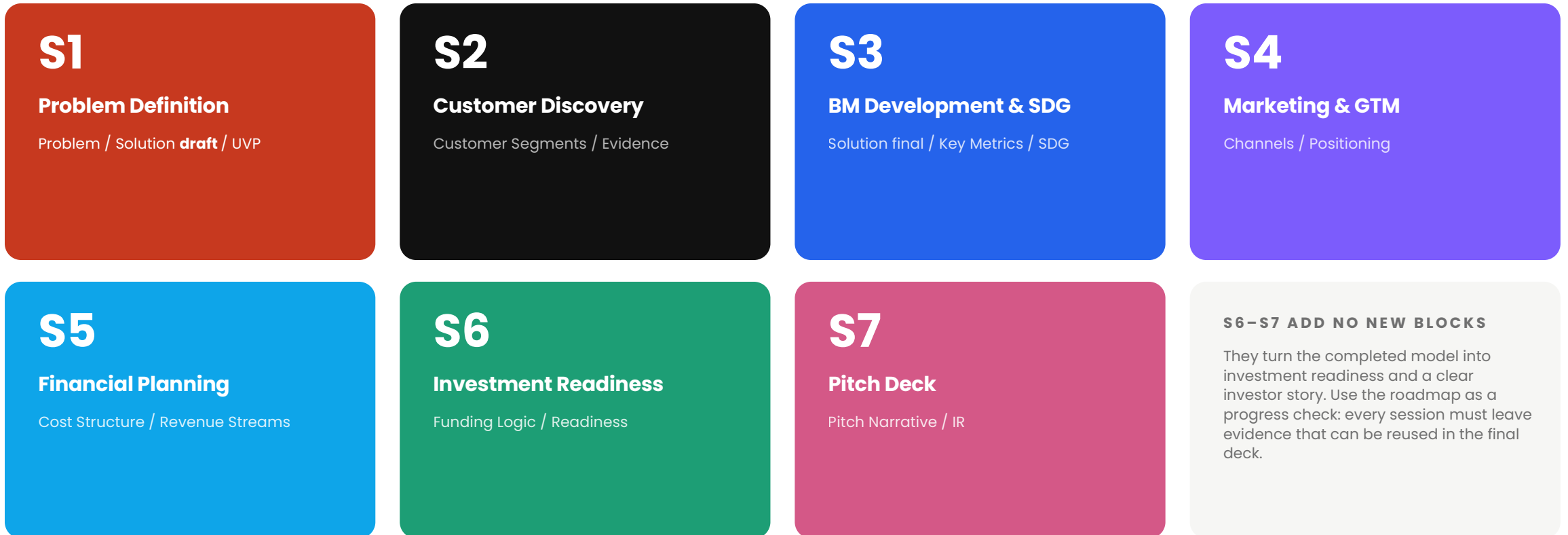
PROBLEM DEFINITION

Problem Definition · **Session 1**

KOICA PMC

Seven Sessions Build One Business Model

S1–S5 complete the 9 blocks. S6–S7 turn the model into investment readiness and investor communication.



**9 BLOCKS BUILT
ACROSS S1–S5**

Problem

Solution

UVP

Segments

Metrics

Channels

Costs

Revenue

Advantage

Lean Canvas Build Map

S1–S5 complete the 9 blocks. S6–S7 translate the completed model into investor communication.

01

Problem

Top 3 pains Existing alternatives

TODAY

02

Solution

Top features · MVP direction

TODAY · DRAFT

03

Unique Value Proposition

Clear promise Why now / why you

TODAY

04

Unfair Advantage

Hard-to-copy edge

05

Customer Segments

Early adopters Decision makers

06

Key Metrics

Learning · impact metrics

07

Channels

Reach, conversion, GTM path

08

Cost Structure

Fixed / variable costs · Runway logic

09

Revenue Streams

Pricing and capture · Revenue model

TODAY Write Problem and UVP. Draft the Solution — S2 validates it with customers, S3 confirms it.

What This Session Produces

Every session leaves participants with a usable BM artifact.

LESSON GOAL Turn an idea into a testable problem, MVP direction, and first value promise.

01

Problem Statement

A specific customer pain and business context.

This becomes the first sentence of the business model story.

02

Problem Hypothesis

A testable belief about who has the pain.

This decides who must be interviewed before building more.

03

MVP Direction

The smallest learning test to run first.

This defines the first small test, not the first full product.

04

UVP Draft

A first value promise linked to the pain.

This links the pain to a first promise customers can understand.

PART 1

Problem Before Solution

Start from pain, not product.

Problem-First Beats Solution-First

Do not force the market to care about your product.

RISK

Solution-First

Starts with features, then searches for a customer problem.

BETTER DIRECTION

Problem-First

Starts with repeated customer pain, then designs the smallest test.

Nigerian Startups Started With Pain

Each case begins with a specific, costly problem that was painful before it became a big company story.

01 Paystack

Online merchants and developers were losing sales because payment acceptance was slow, unreliable, and hard to integrate. The pain was not abstract fintech; it was failed revenue capture.

02 Flutterwave

Merchants operating across African markets faced fragmented payment methods, settlement friction, and corridor-by-corridor complexity. The pain repeated across borders.

03 Moniepoint

MSMEs needed banking and transaction tools that matched daily merchant workflows: receiving money, tracking sales, handling agents, and keeping business moving.

04 LifeBank

Hospitals needed reliable access to blood and medical supplies where delay could create severe health risk. The problem had urgency, measurable consequence, and clear users.

LESSON LENS Read the case as a decision: what pain was real, what evidence mattered, and what can transfer to the team's own model?

How to Read a Startup Case

Use cases to identify the original pain, not just the final valuation.

01 Paystack

Read the founder story backwards: the Stripe exit matters less than the starting pain. Nigerian merchants and developers needed reliable online payment acceptance.

Public cue: Stripe acquired Paystack for \$200M+; use this to show how a narrow payment pain became infrastructure.

02 Flutterwave

Look for the repeated merchant pain across borders. The useful insight is fragmented payment acceptance, settlement, and corridor complexity.

Public cue: Flutterwave describes its platform as payment infrastructure for businesses across emerging markets.

03 Moniepoint

Study the first customer with precision. MSME banking became stronger because the target user, distribution problem, and transaction behavior were concrete.

Public cue: Moniepoint crossed unicorn valuation; use it to discuss MSME transaction activity, not generic fintech hype.

04 LifeBank

Focus on urgency. Blood and medical-supply logistics are not nice-to-have services; delays can create severe health risk.

Public cue: LifeBank positions itself around cold-chain delivery of blood products to healthcare facilities.

Problem-Led Cases Beyond Payments

Sector examples that help students see problem definition outside fintech.

01 Andela

The pain was not 'online education.' African developers lacked credible pathways into global technology work.

MVP / Product: talent screening, training, and placement pathway connecting African developers to global tech teams.

02 Spiro

Okada riders and mobility users face fuel-cost pressure and asset reliability issues in daily income generation.

MVP / Product: electric motorcycle access, battery swapping, and financing that reduce daily rider operating cost.

03 Thrive Agric

Smallholder farmers lacked finance, inputs, technical support, and predictable market access.

MVP / Product: bundled input financing, advisory support, and market linkage for smallholder farmers.

04 Bokku Mart

During inflation, affordable groceries and predictable retail access became a daily household pain.

MVP / Product: affordable grocery retail and neighborhood access model for price-sensitive households.

When the Problem Is Operationally Urgent

Urgency is easier to teach when the consequence is visible in time, safety, or daily income.

01 LifeBank

Blood may exist in the city, but if it does not reach hospitals fast enough, the value is lost.

MVP / Product: cold-chain blood and medical-supply delivery network for hospitals.

02 Terra Industries

Critical infrastructure protection is a security and operational continuity problem before it is a defense-tech product.

MVP / Product: infrastructure risk monitoring and rapid-response coordination workflow.

03 Reel Fruit

Fresh fruit processing and distribution require quality control, supply consistency, and retail execution.

MVP / Product: quality-controlled fruit processing and reliable retail distribution system.

04 MedPlus Pharmacy

Pharmacy retail depends on availability, trust, location, and inventory discipline, not only product variety.

MVP / Product: pharmacy retail access, inventory availability, and trusted fulfillment system.

Where Problems Come From

Useful problems appear in repeated behavior.

01

Customer

A user repeatedly struggles to complete a job.

02

Industry

A system creates delay, cost, or unfair access.

03

Household

Daily life reveals affordability or convenience pain.

04

Founder

Personal pain becomes useful only when others share it.

A Real Problem Has Consequences

If nothing painful changes, the problem is probably weak.

01

Pain

Stress, confusion, frustration, or anxiety.

02

Cost

Money, time, labor, or missed revenue.

03

Risk

Safety, trust, compliance, or reliability exposure.

04

Delay

A process takes too long to complete.

05

Waste

Resources are lost or underused.

06

Exclusion

A group cannot access a needed service.

Needs Are Deeper Than Wants

Customers request wants; founders interpret needs.

RISK

Want

The specific thing customers ask for.

BETTER DIRECTION

Need

The underlying job, pain, or progress they seek.

Paystack: Minimum But Viable

A narrow developer API was enough to test the pain.

CASE STUDY

Paystack

Read the case as a decision: what pain was real, what evidence mattered, and what can transfer to the team's own model?

01 Problem

Online payment setup was slow, unreliable, and revenue-critical for merchants trying to sell online.

02 Minimum

The first useful beachhead could be a narrow payment API and developer-friendly integration path, not a full financial platform.

03 Viable Signal

Developers integrate only when the pain is real. Merchant revenue and failed transactions created a strong reason to try.

04 Outcome

The later Stripe acquisition matters because the early beachhead became trusted payment infrastructure, not because the MVP was large.

Flutterwave: Infrastructure Beachhead

The beachhead was payment infrastructure, not every finance product.

CASE STUDY

Flutterwave

Read the case as a decision: what pain was real, what evidence mattered, and what can transfer to the team's own model?

01 Problem

Cross-border and multi-market payment acceptance created friction for merchants that wanted to collect, settle, and expand across African markets.

02 Minimum

A unified payment API and merchant infrastructure layer could test whether businesses needed one practical payment operating layer.

03 Viable Signal

Payment collection directly affected revenue, so adoption could be observed through merchant usage rather than survey interest.

04 Outcome

The company story became pan-African infrastructure, but the lesson point is the first beachhead: repeated merchant payment friction.

Five Conditions of a Good Problem

Use this before building anything.

01

Payable

Big enough for someone to pay.

02

Scalable

Can support a real business model.

03

Urgent

Pain matters now, not someday.

04

Alternative Exists

People already use workarounds.

05

Customer-Named

Customers describe it in their own words.

Make the Problem Testable

Specific wording creates testable hypotheses.

[Customer] struggles with **[problem]** when **[situation]**, causing **[impact]**.

01

Customer

Who feels it?

02

Problem

What hurts?

03

Situation

When does it happen?

04

Impact

What does it cost?

PART 2

Hypothesis and MVP

Turn the problem into a learning test.

Four Hypotheses Behind Every Idea

A business model changes when one belief is false.

01

Customer

Who has the pain?

02

Problem

What pain repeats?

03

Solution

What could remove it?

04

Value

Why would they care enough?

Customer Problem Hypothesis

Make hypotheses observable.

We believe **[customer]** experiences **[problem]** often enough to **[behavior / payment]**.

01

Customer

Target group

02

Problem

Repeated pain

03

Frequency

How often

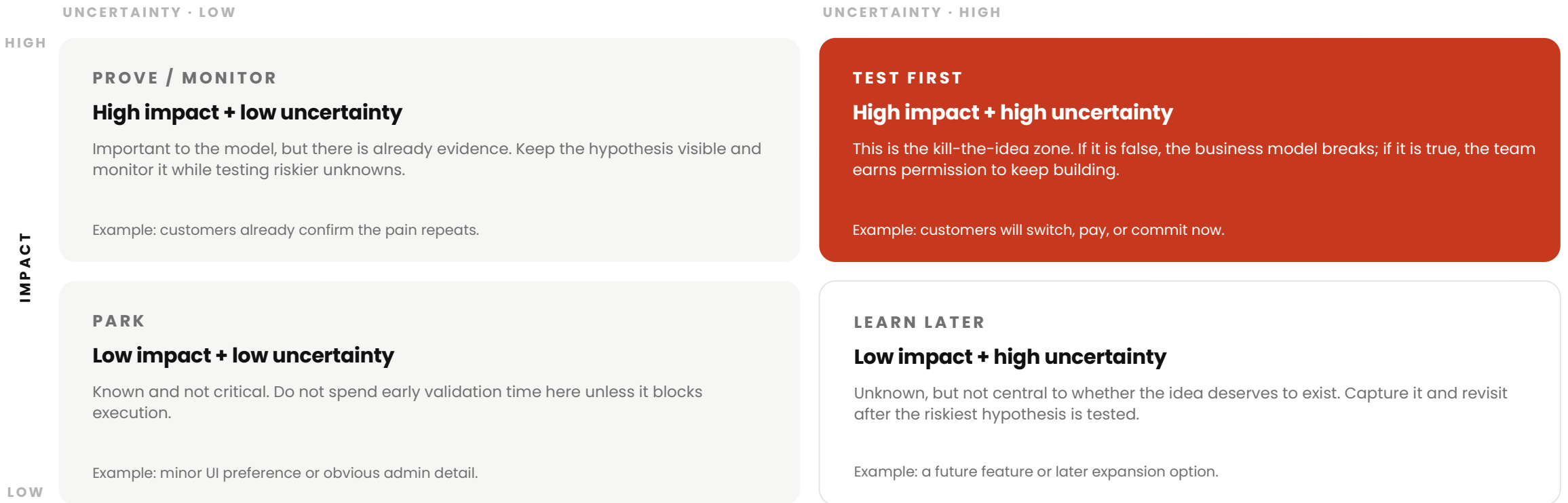
04

Signal

Behavior or payment

Rank What Could Kill the Idea

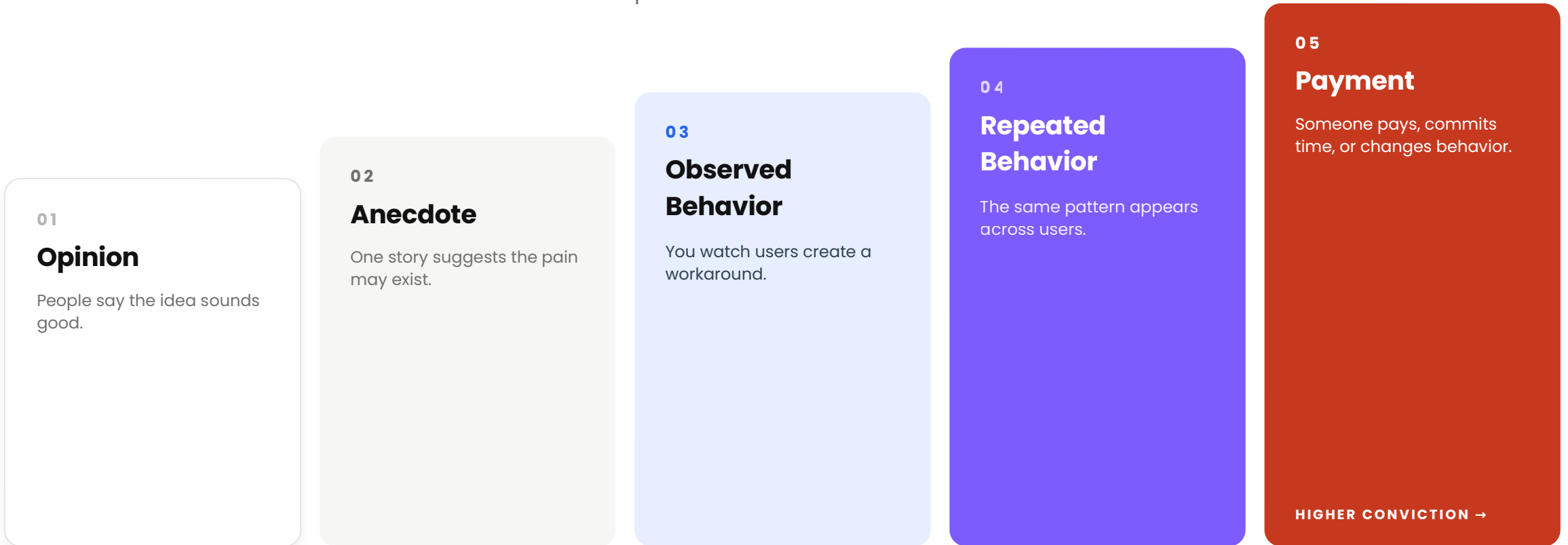
Prioritize hypotheses by impact and uncertainty before choosing the first MVP test.



RULE Run the smallest test that can move one hypothesis out of the TEST FIRST quadrant.

Evidence Has Levels

Conviction increases as evidence moves from words to repeated behavior and commitment.



LESSON POINT Do not treat all evidence equally. Prioritize tests that create observable behavior, repeated behavior, or commitment.

MVP Means Minimum Viable Learning

It is not a small final product.

01

Minimum

Smallest version that can teach you.

02

Viable

Real enough for the user to react.

03

Learning

Evidence that changes the next decision.

How Minimum Was the MVP?

The first test can be surprisingly small if it still creates real behavior.

CASE STUDY

Four minimum tests

Read the case as a decision: what pain was real, what evidence mattered, and what can transfer to the team's own model?

01 Dropbox

Minimum: a demo video. Viable: people understood the workflow and signed up before the full product existed.

02 Zappos

Minimum: shoe photos from local stores. Viable: customers placed real orders, proving willingness to buy shoes online.

03 Airbnb

Minimum: a simple listing page and air mattresses. Viable: guests paid for a real stay, proving the substitute lodging behavior.

04 Stripe / Paystack

Minimum: a narrow developer integration. Viable: developers and merchants had enough payment pain to adopt and test.

MVP Is Not Prototype or Pilot

Each tool answers a different question.

RISK

Prototype / Pilot

Can users understand it? Can it work in a small setting?

BETTER DIRECTION

MVP

Should we keep building based on real evidence?

What Should the First MVP Test?

Test the riskiest hypothesis, not the easiest feature.

01

Pain Intensity

Does the pain matter enough?

02

Willingness to Try

Will users change behavior?

03

Willingness to Pay

Will users commit resources?

04

Workflow Fit

Can it work inside real routines?

Minimum Viable Means the Smallest Real Signal

The first test should be minimum in build effort, but viable enough to trigger real behavior.

01 Dropbox

Minimum: demo video. Viable: users understood the workflow and signed up before a full product existed.

Learning signal: demand for the workflow, not proof of full product readiness.

02 Zappos

Minimum: photos of shoes from stores. Viable: customers placed real orders.

Learning signal: willingness to buy shoes online before inventory was built.

03 Airbnb

Minimum: simple listing page and air mattresses. Viable: guests paid for a real stay.

Learning signal: strangers would pay for alternative accommodation.

04 Paystack-style API Beachhead

Minimum: narrow payment integration. Viable: developers and merchants had revenue-linked pain.

Learning signal: payment acceptance was urgent enough to integrate.

PART 3

Value Proposition Bridge

Turn pain into an early value promise.

Painkiller Beats Vitamin

Urgent pain creates stronger pull than nice-to-have value.

RISK

Vitamin

Nice to have; weak urgency; easy to postpone.

BETTER DIRECTION

Painkiller

Painful, frequent, costly, and urgent.

Value Starts From Customer Pain

Do not describe features before understanding the pain.

01

Pain Reliever

What friction does the product remove?

02

Gain Creator

What new benefit does it add?

03

Alternative

What does the customer use today?

04

Differentiator

Why is this better for this user?

What is Value Proposition?

Pain Relievers & Gain Creators in Action

PAIN RELIEVER

Airbnb

Hotels are expensive; unfamiliar stays feel risky → Verified host reviews + simple booking make any destination feel safe

Uber

Hard to hail a cab; pricing is opaque → One-tap ride request + upfront fare shown before you book

Slack

Email CC chains; messages fall through cracks → Channel-based real-time messaging eliminates email dependency

Flutterwave

Cross-border payments slow, costly, fragmented → Single API to process payments across 30+ countries

GAIN CREATOR

Tesla

Just want an eco-friendly car → Best-in-class performance + OTA updates improve the car after purchase

Notion

Just want to organize notes → Team collaboration + database + wiki in one workflow

LinkedIn

Want a slight edge in job hunting → See who viewed profile + InMail to decision-makers

Zoom

Just need meetings to work → Webinars, recording, captions, AI summaries beyond the meeting

Pain Relievers and Gain Creators Need Different Proof

Use familiar cases to show whether the value removes a painful workaround or creates a new attractive gain.

01 Airbnb

Pain reliever: travelers needed cheaper, available accommodation when hotels were expensive or unavailable.

Proof to seek: real paid stays, host supply, repeat booking, and trust mechanisms.

02 Uber

Pain reliever: riders needed more reliable, transparent transportation than street hailing or dispatch uncertainty.

Proof to seek: repeat rides, driver supply, wait-time reduction, and willingness to pay.

03 Slack

Pain reliever and gain creator: teams wanted less fragmented communication and searchable work context.

Proof to seek: team activation, daily use, retained workspaces, and message volume that improves workflow.

04 Flutterwave

Pain reliever: merchants needed payment acceptance across fragmented African markets.

Proof to seek: merchant activation, transaction volume, successful settlement, and corridor expansion.

Solution Statements Should Stay Close to the Pain

Use company examples to practice turning a problem into a solution statement without over-describing features.

01 Flutterwave

Problem: fragmented African payment acceptance.

Solution statement: one payment infrastructure layer helps businesses collect, settle, and expand across markets.

02 Tesla

Problem: EV adoption needed product desirability, charging confidence, and clean-energy ecosystem logic.

Solution statement: integrated EV and energy products make clean mobility more practical and aspirational.

03 Zoom

Problem: remote communication tools were unreliable or hard to use.

Solution statement: simple, reliable cloud video meetings reduce friction for teams, schools, and organizations.

04 Paystack

Problem: Nigerian online merchants lost revenue when payments failed.

Solution statement: developer-friendly payment acceptance lets businesses start collecting online faster.

Draft the First UVP

The UVP must connect to the problem.

We help **[WHO]** do **[WHAT]** better by **[HOW]**, unlike **[ALTERNATIVE]**.

01

Who

Customer

02

What

Job or progress

03

How

Solution angle

04

Alternative

Current workaround

BRIDGE · SESSION 2

Next Session — Customer Discovery

Customer Discovery tests whether your problem hypothesis survives real customers.